

MinimizerModule — Algorithm Guide

(Downloadable)

Purpose (one line): A compact, input-agnostic, high-performance algorithmic module to compute constrained isoperimetric minimizers (volume-preserving minimal surfaces / cap-like shapes) inside arbitrary domains. The module is strictly for data processing—no rendering or color concerns.

1. Design goals

- **Input-agnostic:** accept point clouds, analytic SDFs, meshes, or canonical containers.
 - **Encapsulated API:** clear functions for domain mapping, SDF construction, constrained minimizer computation, extraction, and serialization.
 - **Fast & scalable:** quick coarse envelope estimation, narrow-band methods, worker/CPU/GPU-friendly components.
 - **Deterministic & testable:** deterministic algorithms with repeatable numeric parameters and validation tests.
-

2. Core concepts & representations

- **Domain Ω :** the container in which minimizers must lie. Represented as either:
 - Analytic SDF: $\phi(x)$ (callable), or
 - Point cloud / mesh: converted to voxel SDF via fast envelope then narrow-band SDF.
 - **Minimizer (unknown):** the zero level-set $\Sigma = \{ x \mid \phi_{\min}(x)=0 \}$ of an SDF $\phi_{\min}$ inside Ω that minimizes surface area for fixed enclosed volume.
 - **Narrow-band SDF:** grid-based signed distance maintained only near interface ($\pm B$ voxels) for speed.
-

3. Public API (signatures)

```
// Initialize module (returns instance)
createMinimizerModule(opts?: { gridSize?: number, bandwidth?: number,
  backend?: 'cpu' | 'webworker' | 'gpu' }) : MinimizerInstance

// Domain setup
minimizer.setDomain(domain: Domain) : Promise<void>
  // Domain: { type: 'analytic', sdf: (x) => number } | { type: 'pointcloud',
  points: Float32Array } | { type: 'mesh', verts, tris } | { type: 'canonical', name,
  params }

// Quick envelope / bounding frame
const envelope = await minimizer.computeEnvelope(options?)

// Convert domain to voxel SDF (narrow-band)
```

```

await minimizer.buildVoxelSDF(resolution:number)

// Compute constrained minimizer(s)
// options: {initialVolume?, targetVolume?, contactAngle?, volumeTolerance?,
maxIters?, solverParams?}
const result = await minimizer.solveMinimizer({targetVolume, contactAngle,
initType:'cap'|'ball'|'random'})

// Sweep over parameters (batch)
const family = await minimizer.sweepParameters([targetVolume,
contactAngle}, ...], options)

// Extract mesh or analytic SDF
const mesh = minimizer.extractMesh(isovalue=0.0)
const sdf = minimizer.exportSDF() // narrow-band grid

// Persistence
minimizer.serialize() -> JSON
minimizer.load(serialized)

```

4. Major components & flow

1. Envelope estimation (fast):

2. If domain is point cloud: compute convex hull for coarse envelope (2D/3D) or support-sample along m directions for directional envelope. Use QuickHull ($O(n \log n)$) or rotating calipers for 2D.
3. Optionally compute alpha-shape (concave hull) when concavity matters (slower).
4. Output: closed mesh or analytic container.

5. Voxelization & narrow-band SDF:

6. Create axis-aligned bounding box around envelope, pad by margin M .
7. Allocate voxel grid of resolution N^d ($d = 2/3/4$ as needed) but only store narrow-band voxels within $\pm B$ voxels of interface to save memory.
8. Compute unsigned distance with fast Euclidean distance transform and assign sign via inside-test (ray cast or winding number).

9. Initial minimizer guess:

10. Options: centered ball (radius from target volume), cap (intersection of ball & half-space), or analytic SDF supplied by caller.
11. Represent guess as narrow-band SDF ϕ_0 .

12. Solver — Volume-preserving mean-curvature flow (level-set):

13. PDE (continuous form): $\partial\phi/\partial t = - (H - \lambda) |\nabla\phi|$, where $H = \text{div}(\nabla\phi/|\nabla\phi|)$ and λ is Lagrange multiplier enforcing volume.
14. Discretization: semi-implicit / explicit time stepping on narrow-band grid. Compute curvature via centered finite differences.
15. Volume constraint: compute instantaneous volume change for candidate step; solve for λ (linear approximation) or apply augmented-Lagrangian.
16. Reinitialization: periodically reinitialize ϕ to signed distance (fast-marching or PDE-based reinit) to preserve numeric stability.

17. Contact / Young's law enforcement:

18. The free-boundary condition on $\partial\Omega$ is $\mathbf{n}_\Sigma \cdot \mathbf{n}_{\partial\Omega} = \cos \theta$.

19. Implementations:

- **Penalty approach:** add boundary penalty $E_c = \alpha \int_{\Sigma \cap \partial\Omega} (\mathbf{n}_\Sigma \cdot \mathbf{n}_{\partial\Omega} - \cos \theta)^2 dA$ and include its variational term in update.
- **Projection approach:** after each time step, at narrow-band voxels adjacent to $\partial\Omega$, locally rotate projected normal to satisfy dot-product constraint and adjust ϕ accordingly.
- **Hybrid:** use penalty for stability and projection for accuracy.

20. Nesting & multiple minimizers:

21. If multiple minimizers with parent-child relations are desired, solve them sequentially or simultaneously with masked domain support:
- For child constrained to lie inside parent: apply mask $\phi_{\text{child}} = \max(\phi_{\text{child}}, \phi_{\text{parent}} + \epsilon)$ to prevent child from venturing outside parent.
 - For interacting minimizers (multiple components), maintain separate ϕ_i and optionally include interfacial energies for contact.

22. Extraction & post-processing:

23. Extract zero-level set via marching cubes (3D) or marching squares (2D). For 4D, extract 3D slices or hyper-surface extraction tools.
24. Smooth mesh, simplify triangles, compute normals, and compute surface area and volume for validation.

5. Numerical parameters & recommendations

- **Grid resolution** N : choose so that typical feature size ≥ 8 voxels. For interactive/fast tasks choose 128^3 ; for high-fidelity exports use 256^3 +
- **Band width** B : 6–12 voxels.
- **Time step** Δt : CFL-limited; start with $\Delta t \approx 0.2 * dx$ and adapt.
- **Reinit frequency:** every 10–20 iterations.
- **Stop criteria:** $\max \text{Iters}$ or $|\Delta \text{Volume}| < \text{tolVolume}$ and $\max |\Delta \phi| < \text{tolPhi}$.
- **Penalty weight α (contact):** scale by grid spacing: $\alpha \approx C / dx$ with $C \sim 0.1$ –10.

6. Complexity & performance

- **Envelope estimation:** $O(n \log n)$ for hulls; support sampling $O(n \cdot m)$.
 - **SDF build (narrow-band):** $O(k)$ where k is number of voxels in band.
 - **Level-set solver:** $O(k \cdot \text{iters})$.
 - **Mesh extraction:** $O(k)$.
 - **Parallelization:** narrow-band updates highly parallel—good fit for GPU/WebGPU or multi-threaded workers.
-

7. Testing & validation

- Implement unit cases with analytic solutions:
 - Unconstrained ball: verify that solver returns sphere for free space.
 - Spherical cap in ball: compare numeric area/volume vs closed-form cap formula.
 - Contact angle extremes: $\theta=90^\circ$ (orthogonal), $\theta \rightarrow 0$, $\theta \rightarrow 180^\circ$.
 - Regression tests: deterministic RNG seeds, fixed solver params, compare extracted area/volume within tolerances.
-

8. Serialization & interchange

- Export SDF narrow-band as compact JSON with bounding box, grid spacing, index-to-coordinate mapping, and signed distances array.
 - Export meshes as glTF/OBJ/STL with metadata (targetVolume, contactAngle, solverParams).
-

9. Implementation notes & language suggestions

- **JS/TS (Web):** implement envelope & narrow-band SDF; heavy solver in WebWorker; for high-performance use WebGPU compute shaders.
 - **C++ / Python:** use for offline heavy processing. Libraries: CGAL (hulls), libigl (meshes), VTK for extraction.
-

10. Minimal pseudocode (main flow)

```
module = createMinimizerModule(opts)
module.setDomain(domain)
envelope = module.computeEnvelope()
module.buildVoxelSDF(res)
phi0 = module.initialGuess({method: 'cap', targetVolume})
result = module.solveMinimizer({targetVolume, contactAngle, maxIters})
mesh = module.extractMesh()
save(mesh)
```

11. Deliverables expected from a dev implementing this module

1. Well-documented API matching signatures above.
2. Unit tests for analytic cases.
3. Example scripts: point-cloud → envelope → SDF → solve → export mesh.
4. Benchmarks demonstrating scalability and speed on representative datasets.

End of MinimizerModule — Algorithm Guide.